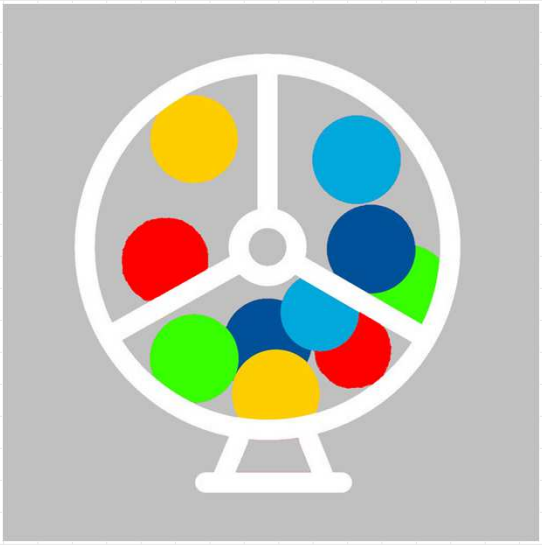
Statistics
Basic elements

just what I'll need
(not for statisticians!)

Vittorio Maniezzo - Universita di Bologna

1



2

Population and sample

Population: the entire set of data, individuals, objects or results of interest.

- Often too large to be fully analyzed
- It can be real or hypothetical (e.g. the results of an experiment repeated countless times)

Sample: a subset of the population.

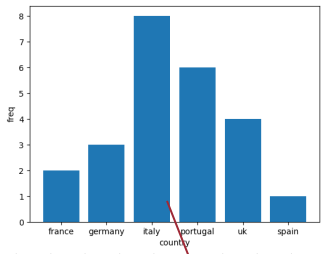
- A sample can be *random* (each member is equally likely to be extracted from the population) or *reasoned choice* (non-probabilistic).
- Random selection seeks to ensure that the sample is representative of the population.

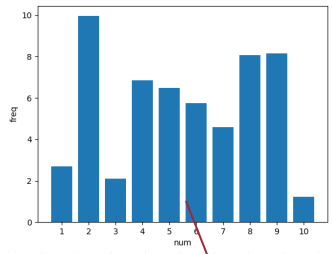
Variables

Variables are the quantities measured in a sample. They can be:

- Quantitative (numeric)
  - *Continuous*: Quantified on a continuous scale (e.g. height of people in the classroom). However two values are set, all intermediate values can be assumed.
  - *Discrete*: quantified with counts (e.g. number of people in the classroom). For any value, there is a whole interval with the value is the center, where no other value can be assumed.
- Categorical
  - *Nominal*: values identify categories, but quantities do not make sense (e.g. gender, nationality).
  - *Ordinal*: values allow sorting, but ranges between values can be variable (e.g. employment levels, hierarchies at work).

Variables





Categorical, nominal

Quantitative, discrete

Measurement levels

Nominal scales: values identify **categories**, quantities do not make sense. Only *relationship*: identity, only *operation* allowed: counting

Ordinal scales: values allow **sorting** but ranges between values can vary. Asymmetric and transitive *relations*, it is *not possible to quantify the differences* in intensity between the observations.

Interval scales: the measurements are **continuous** with equal intervals between points; **zero is arbitrary** (e.g. weather, temperature Fahrenheit or Celsius). The zero does not indicate the total absence of the quantity being measured, the ratio between pairs of values is not possible (a temperature of 80 degrees is not twice as high as one of 40 degrees)

Ratio scales: all previous properties and also **a natural zero** (e.g. height, distance, speed, age, weight, income, Kelvin temperature).

Parameters and statistics

Parameters: quantities that describe the characteristics of a population. They are usually not known, and we want to make a statistical inference about the parameters.

Descriptive statistics: quantities and techniques used to describe the characteristics of a data set, e.g., mean, standard deviation, box-plot, …

Inferential statistics: techniques to analyze samples and generalize them to the population

Sample error: difference between sample statistics and the values of the corresponding population parameters

Descriptive Statistics

Frequency distributions

An (empirical) Frequency Distribution or *Histogram* for a continuous variable has a count of observations, grouped into predefined classes or groups

A Relative Frequency Distribution has the corresponding proportions of observations within the classes

A bar chart presents frequencies for a categorical variable

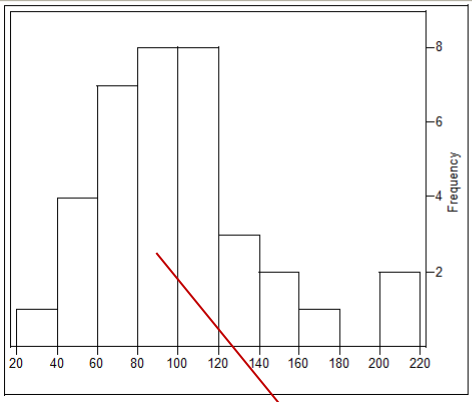
Example – Highway speeds

Speed camera readings of vehicles traveling a stretch of motorway, measured in km / h.

121	82	100	151	68	58
95	145	64	201	101	163
84	57	139	60	78	94
119	104	110	113	118	203
62	83	67	93	92	110
25	123	70	48	95	42

Frequency distribution

Distributions



Quantiles		
100.0%	maximum	203.00
99.5%		203.00
97.5%		203.00
90.0%		154.60
75.0%	quartile	118.75
50.0%	median	94.50
25.0%	quartile	67.25
10.0%		54.30
2.5%		25.00
0.5%		25.00
0.0%	minimum	25.00

Moments	
Mean	98.277778
Std Dev	40.380767
Std Err Mean	6.7301278
upper 95% Mean	111.94066
lower 95% Mean	84.614892
N	36

Distribution, we will elaborate on this

11

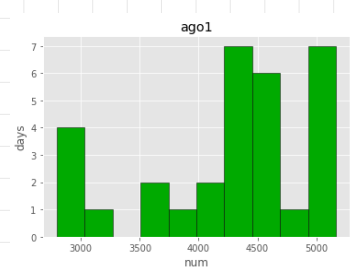
Example – traffic counts

ago1	ago2	set1	set2	ott1	ott2
5012	5334	4820	5149	5425	5797
4948	5107	5062	5159	4433	4642
5077	5259	4182	4532	5512	5856
4960	5037	3177	3235	5641	5926
5010	5304	5068	5292	5751	5920
4170	4371	5004	5178	5468	5804
2916	2959	5211	5287	5493	6061
4442	4545	4960	5174	4996	5327
4301	4391	5383	5534	3510	3552
4931	4975	4355	4511	5524	5730
4533	4703	3090	3290	5472	5751
4438	4494	5215	5420	5776	6057
3707	3947	5203	5442	5560	5785
2794	2812	5602	5739	5864	6166
3148	3184	5445	5576	4941	5181
4086	4240	5562	5775	3892	4065
4355	4463	4752	5172	5459	5689
4272	4437	3247	3499	5507	5699
4389	4614	5855	6088	5703	5963
3546	3851	5630	5749	5487	5836
2827	2917	5614	5921	5738	6166
4505	4594	5484	5713	5002	5277
4598	4653	5599	5880	3542	3663
4646	4762	5010	5290	5533	5787
4406	4500	3421	3639		
4586	4726	5454	5755		
3801	4103	535	520		
2845	2869	7626	7321		
4919	4728	5556	5690		
4663	4885	5593	6071		
5167	5405				

```python
import numpy as np
import pandas as pd
from scipy import stats            # to be used later
import matplotlib.pyplot as plt
import os

df = pd.read_csv('traffico16.csv') # dataframe (series)
npa = df['ago1'].to_numpy()        # numpy array

plt.hist(npa, bins=10, color='#00AA00', edgecolor='black')
plt.title(df.columns[0]);plt.xlabel('num');plt.ylabel('days')
plt.show()
```



12

Relative frequency table

num	Frequence	Relative Frequence	Cumulative rel. freq.
2600-2926	4	0.13	0.13
2926-3189	1	0.03	0.16
3189-3453	0	0.00	0.16
3453-3717	2	0.06	0.23
3717-3980	1	0.03	0.26
3980-4244	2	0.06	0.32
4244-4508	7	0.26	0.58
4508-4771	6	0.16	0.74
4771-5035	1	0.19	0.94
5035-5300	7	0.06	1.00
Totale	**31**	**1.000**	

Python:
```
res = stats.relfreq(npa, numbins=10)
print(res[0])
```


measures of central tendency

The central tendency measures indicate in which area of the range of permissible values the data are located. Common measures are:

1. The arithmetic mean
2. The median
3. The mode

The mean (average)

Let $x_1, x_2, x_3, \ldots, x_n$ *be* the measured values of a random variable $\mathbf{X}$, from a sample of cardinality $\boldsymbol{n}$.

The ***arithmetic mean*** is defined as:

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i = \sum_{i=1}^{n} p(x_i)x_i$$

In Excel: MEDIA(dati)
In Python: np.mean(npa)

Example

Some of the speeds measured on a motorway section are:

151, 124, 132, 170, 146, 124, 113.

The average is

$$\bar{x} = \frac{(151 + 124 + 132 + 170 + 146 + 124 + 113)}{7} = 137.14$$

Median and Mode

- *n* sample data are organized by increasing values, then the median is
  - The middle value if n is odd
  - The average between the two middle values of half if n is evan

In Excel: MEDIANA(dati)

*In Python: np.*median(npa)

- The mode is the most frequently detected value.

In Excel: MODA(dati)

In Python: stats.mode(npa)

Example

N odd

The speeds seen before, ordered, are:

113, 124, 124, 132, 146, 151, 170.

The median is the middle value: 132.

Two travelers were driving at 124 Km/h, so the mode is 124.

N even

If you want to book a hotel for a skiing holiday in the mountains, you ask for quotes from six hotels. Quotes are:

366, 327, 274, 292, 274, 230.

Rearranged by ascending order: 230, 274, 274, 292, 327, 366.

The median is halfway between the two central values: (274+292)÷2 = 283.

Two hotels asked for the same amount, mode is 274.

Mean and median

If the sample contains very high or very low values, the average tends to be distorted..

The median is not affected by very large (or very small) values, so it is a better measure of centrality when the distribution is distorted..

If average=median=mode then the data are called symmetric.

Quartiles and percentiles

Using the same principle of increasing ordering of data and their location, it is possible to define various quantiles (for example, dividing into 4 intervals gives quartiles, and so on).

If you divide into 100 ranges, you get ***percentiles***.

For example, the 75th percentile is the value of the data that, in ascending ordering, has a position such that:

- 75% of data has a lower value (i.e. remains on the left in the sort)
- 25% of data has a higher value (i.e. stays on the right in sorting)
- Note: The median is the 2nd quartile and the 50th percentile

Quartiles and IQR

The median divides a distribution into two halves.

The ***first*** and ***third*** quartiles (denoted Q1 and Q3) are defined as:

- 25% of data is below Q1 (and 75% above Q1),
- 25% of data are above Q3 (and 75% below Q3)

The ***inter-quartile range*** (IQR) is the difference between the first and third quartile: IQR = Q_3- Q_1

Example ordered speeds:

113	124	124	132	146	151	170
		Q_1			Q_3	

Inter Quartile Range (IQR): 151-124 = 27

In Excel: manual (diff. between quartiles)

In Python: `df['ago1'].quantile([0.25,0.5,0.75])`

`stats.iqr(npa)`

Dispersion measures

Dispersion measures characterize how much the sample is scattered around the mean value, how much variable the data are.

Commonly used dispersion measures are:

1. Range
2. Variance and standard deviation
3. Coefficient of variation (or relative standard deviation)
4. Inter-quartile range (seen before)

Range

The Range of the sample is the difference between the largest and smallest value in the sample.

Easy to compute:

- Example speed: min=25, max=203, then range=178 Km/h

Useful for defining scenarios, the best or the worst

Very sensitive to extreme values.

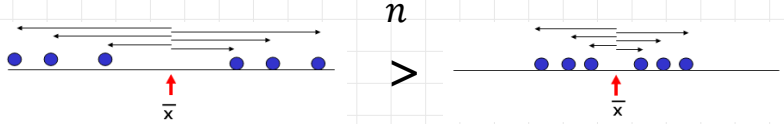
In Excel: MAX(dati)-MIN(dati),
RANGE (ingl) is another thing

In Python: `max(npa)-min(npa)`
range() is another thing

Variance

Variance, σ^2, is the arithmetic mean of the square of deviations from the mean:

$$s^2 = \frac{\sum_{i=1}^{n}(x_i - \bar{x})^2}{n}$$



Note: The estimator is correct if the population average is known, otherwise (only the sample is known) it is necessary to divide by n-1 and not by *n*.

$$\sigma^2 = \frac{\sum_{i=1}^{n}(x_i-\bar{x})^2}{n-1} \leftrightarrow \frac{\sum_{i=1}^{n}(x_i-\bar{x})^2}{n}$$

In Excel: VAR.P(dati), VAR.C(dati)
In Python: np.var(npa)

Standard deviation

The **standard deviation σ** is the square root of variance

$$\sigma = \sqrt{\frac{\sum_{i=1}^{n}(x_i - \bar{x})^2}{n}} \quad \text{oppure } \sigma = \sqrt{\frac{\sum_{i=1}^{n}(x_i - \bar{x})^2}{n-1}}$$

σ has the advantage of having the same unit of measurement as the original variable **x**

In Excel: DEV.ST.P(dati), DEV.ST.C(dati)
In Python: np.std(npa)

Example

Data	*Deviation*	*Deviation²*
151	13.86	192.02
124	-13.14	172.73
132	-5.14	26.45
170	32.86	1079.59
146	8.86	78.45
124	-13.14	172.73
113	-24.14	582.88
Sum= 960.0	Sum= 0.00	Sum= 2304.86

$\bar{x} = 137.14$

$$\sum_{i=1}^{7}(x_i - \bar{x})^2 = 2304.86 \qquad \sigma = \sqrt{\frac{2304.86}{7}} = 18.14$$

Coefficient of Variation

The **coefficient of variation** (CV) or **Relative standard deviation** (RSD) is the standard deviation expressed as a percentage of the mean:

$$CV = \left(\frac{s}{\bar{x}}\right) \times 100\%$$

The CV It is not affected by multiplicative variations from the scale, so it is useful when you want to *compare distributions of variables measured on* ***different scales***

Example:

$$CV = 100 \times \left(\frac{19.6}{137.1}\right)\% = 14.3\%$$

In Python: `stats.variation(npa)`

Python: numpy array descriptions

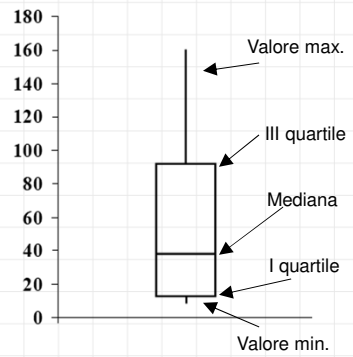
The only command that provides many descriptive statistics for a series (or dataframe column):

```
print(df['ago1'].describe() )
   count       31.000000
   mean      4258.000000
   std        720.147901
   min       2794.000000
   25%       3943.500000
   50%       4438.000000
   75%       4791.000000
   max       5167.000000
   Name: ago1, dtype: float64
```

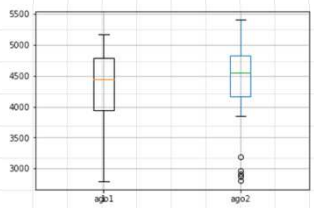
Box-plot

A box-plot is a visual representation of a distribution based on:

- Minimum
- Q1
- Median
- Q3
- Maximum



Useful for comparing large data sets



In Python: `df.boxplot(column=['ago1', 'ago2'])`
`plt.boxplot(npa)`

Example

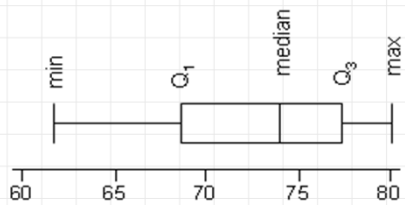
Speed on rural roads:

62, 64, 68, 70, 70, 74, 74, 76, 76, 78, 78, 80

Q_1=(68+70)÷2 = 69,

Q_3=(76+78)÷2 = 77

IQR = (77 – 69) = 8



Comparison example

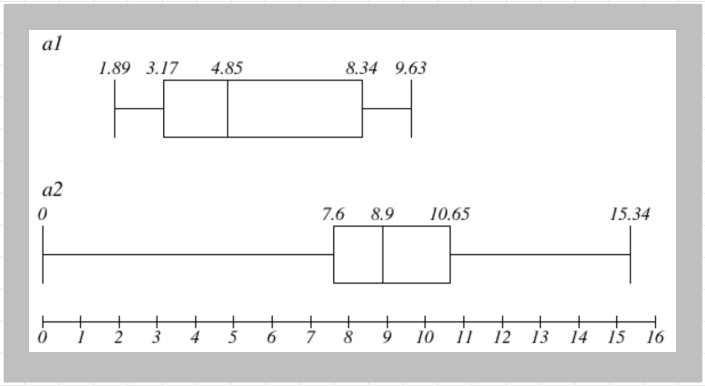
We want to compare two minimization algorithms. Try them both on 12 test problems.

nome	opt	HEU1	HEU2	a1	a2
gapc_0	1931	2117	2059	9.63	6.63
e201600	180659	197864	189087	9.52	4.67
gapd_0	6353	6776	7246	6.66	14.06
gapb_0	1843	1905	1989	3.36	7.92
cesenaGAP	2881	2956	3323	2.60	15.34
gapa_0	1698	1763	1844	3.83	8.60
gapd_4	6185	6493	7115	4.98	15.04
e05100	12681	12976	13887	2.33	9.51
e10200	23307	23749	25452	1.90	9.20
e20400	44879	47000	49052	4.73	9.30
elba	10979	11860	10979	8.02	0.00
e401600	178307	194866	192588	9.29	8.01

Elementary descriptive statistics:

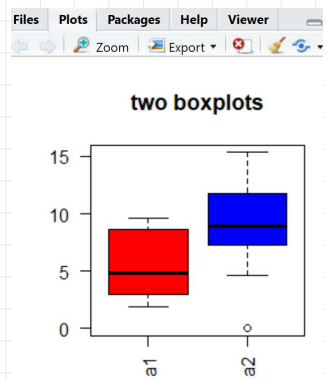
	a1	*a2*
media	5.57	9.02
varianza	8.63	19.12
stdev	2.94	4.37
mediana	4.85	8.90
moda	N/A	N/A
Q1	3.17	7.60
Q3	8.34	10.65
IQR	5.17	3.05
range	7.74	15.34
CV	52.74	48.46

Example: box-plot comparison



Traffic example: in Python

```
a1=c(9.632314863,9.523466863,6.658271683,3.364080304,2.603262756,3.82803298,4
.979789814,2.326314959,1.896425966,4.726041133,8.024410238,9.286791881);
a2=c(6.628689798,4.665142617,14.05635133,7.921866522,15.34189518,8.598351001,
15.03637833,9.510290987,9.203243661,9.298335524,0,8.009220053);
> A = data.frame(a1,a2);
> str(A);
'data.frame':   12 obs. of  2 variables:
 $ a1: num  9.63 9.52 6.66 3.36 2.6 ...
 $ a2: num  6.63 4.67 14.06 7.92 15.34 ...
> boxplot(A$a1,A$a2,
+         main = "two boxplots",
+         at = c(1,2),
+         names = c("a1","a2"),
+         las=2,
+         col=c("red","blue"),
+         border="black",
+         horizontal=FALSE,
+         notch=FALSE)
>
```



Outlier

An **outlier** is an observation with very different value from those of the other data.

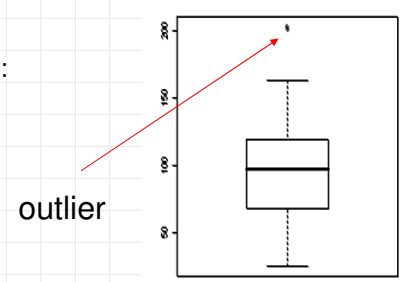
An outlier may be due to a measurement problem or may be indicative of a sub/population with abnormally high or low values.

To represent them in a box-plot, redefine the lower and upper limits(of the lines) as:

Lower limit= Q_1-1.5×IQR

Upper limit= Q_3+1.5×IQR

Data may not reach achieve these values!

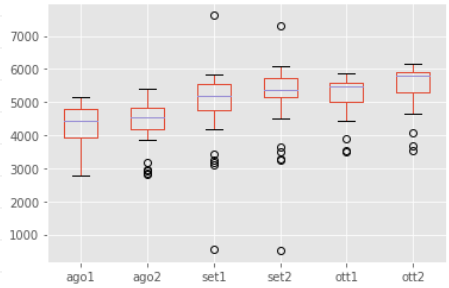


If there are data < limit inf. or > limit sup., they are considered *outliers*.

Outlier

In Python:

```
> df.boxplot(column=['ago1', 'ago2','set1', 'set2','ott1', 'ott2'])
```



Outliers can be disturbing

Descriptions:

- Distorting the average.
- Increasing variability.

Elimination of outliers:

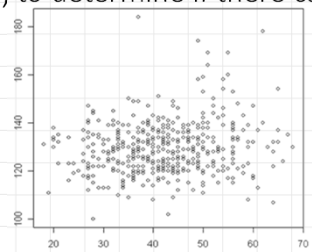
- In a *normal* sample value should always be within 3 SD of the mean.
- Often external values at 1.5-2 SD are discarded a priori.

Scatter-plot

Represents the relationship between two continuous variables

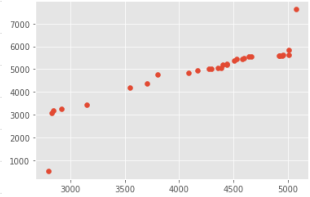
Useful in the early stages of an investigation, to determine if there can be high correlation between the two

Makes outliers evident



In Python:

```
plt.scatter(df['ago1'].sort_values(),df['set1'].sort_values() )
plt.show()
```



Statistical distributions

Measures of center and spread only tell a part of the story.

We often need to look at the shape of the distribution by creating histograms, that show the frequency of values in a data set.

A sample of data will be described by a distribution, i.e., by a function that describes the relationship between observations in the sample space

A distribution is a theoretical curve that models a histogram's shape. We can use these curves to calculate estimated probabilities on which to base our conclusions.

The function will fit the data with a modification of its parameters, such as for example mean and standard deviation

Density functions

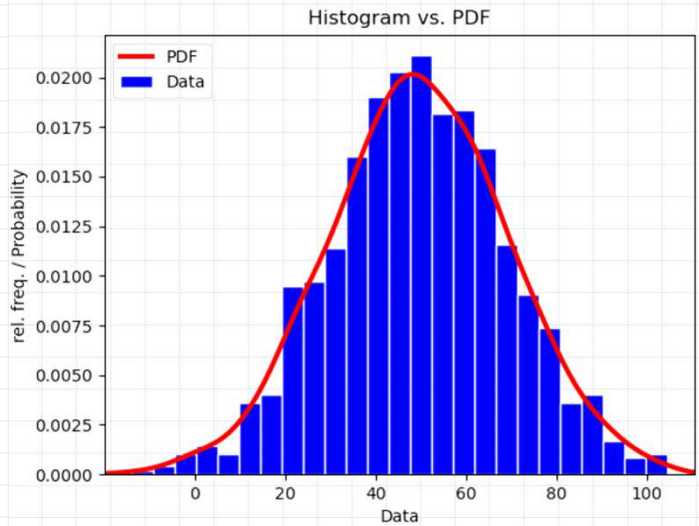
Distributions are often described in terms of their density or density functions.

Density functions are functions that describe how the proportion of data or likelihood of the proportion of observations change over the range of the distribution.

Two types of density functions are probability density functions and cumulative density functions.

- Probability Density function (PDF): represents the probability of observing a given value (only positive values, area below PDF = 1).
  - PDF are defined over continuous variables. The equivalent of a PDF for a discrete distribution is called a probability mass function, PMF.
- Cumulative Density function (CDF): represents the probability of an observation equal or less than a value.

Distribution and PDF



Distributions and PDFs

PDF is typically used to calculate

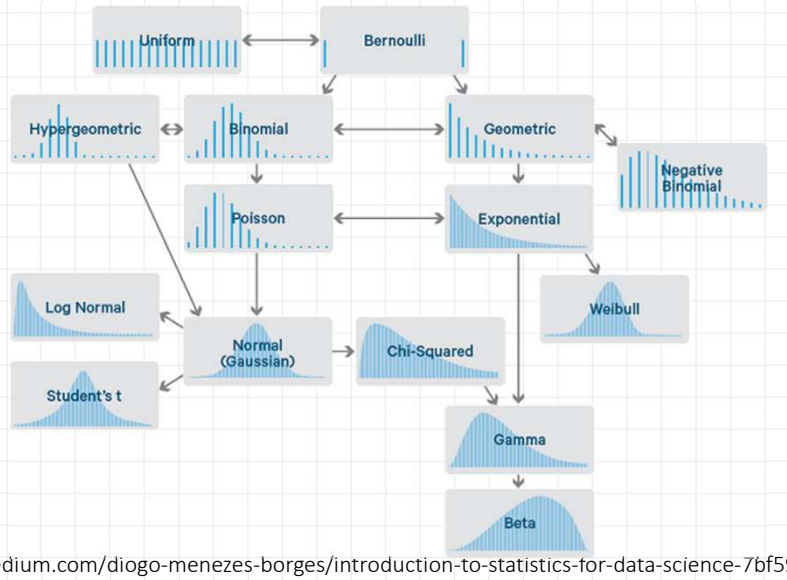
- the likelihood of a given observation in a distribution
- the likelihood of observations in subsets of the distribution's sample space.

Plots of the PDF show the shape of a distribution.

Distributions are often defined in terms of their PDF with their associated parameters.

A CDF represents the cumulative likelihood for an observation and all prior observations in the sample space. It allows us to quickly understand how much of the distribution lies before and after a given value. A CDF is often plotted as a curve from 0 to 1 for the distribution.

Distributions



https://medium.com/diogo-menezes-borges/introduction-to-statistics-for-data-science-7bf596237ac6

Gaussian distribution

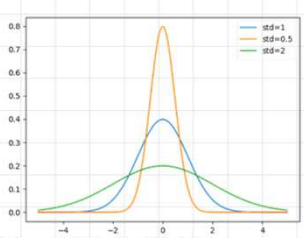
https://en.wikipedia.org/wiki/Normal_distribution

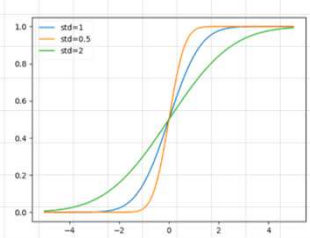
The Gaussian distribution, named after Carl Friedrich Gauss, is the most commonly used PDF. Data from so many fields of study can be described using a Gaussian distribution that the distribution is also called the "normal" distribution.

A Gaussian distribution has two parameters:

- mean μ, the expected value of the distribution.
- standard deviation σ, is the normalized spread of observations from the mean.

$$\text{PDF: } f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$





Gaussian distribution

Gaussian distribution can be generated in many ways: via norm in SciPy, random.normal in numpy, statsmodels, ….
The norm.pdf() generates a Gaussian PDF with parameters sample space, mean, and standard deviation.

```
from scipy.stats import norm
dom = np.arange(-5, 5, 0.001)
mean = 0.0
plt.plot(dom, norm.pdf(dom, mean, 1), label="std=1")
plt.plot(dom, norm.pdf(dom, mean, 0.5), label="std=0.5")
plt.plot(dom, norm.pdf(dom, mean, 2), label="std=2")
plt.legend()
plt.show()
```

```
plt.plot(dom, norm.cdf(dom, mean, 1), label="std=1")
plt.plot(dom, norm.cdf(dom, mean, 0.5), label="std=0.5")
plt.plot(dom, norm.cdf(dom, mean, 2), label="std=2")
plt.legend()
plt.show()
```

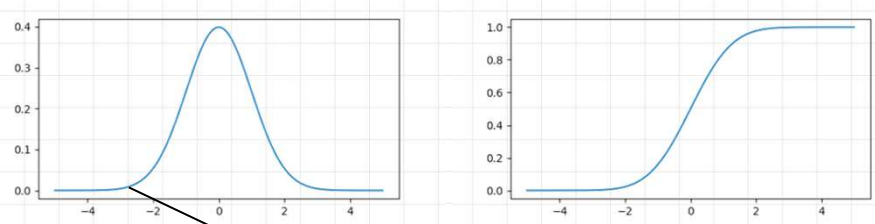

Student's t distribution

https://en.wikipedia.org/wiki/Student%27s_t-distribution

The Student's *t*-distribution (or just *t*-distribution) is named after the pseudonym "Student" of William Gosset. It is very similar to the Gaussian, it is used to describe uncertainty related to estimating population statistics for data drawn from Gaussian distributions when the size of the sample must be taken into account. We will use it to compare the means of two different samples.

The distribution can be described using a single parameter, the number of degrees of freedom, ν.

PDF, *rather complicated*, includes the *gamma function*.



Tails are a bit higher than in the normal

Student's t distribution

SciPy provides tools for working with the t-distribution in the stats.t module. The t.pdf() function can be used to create a Student t-distribution with the specified degrees of freedom.

```
from scipy.stats import t

dom = np.arange(-5, 5, 0.001)
dof = len(dom) - 1
pdf = t.pdf(dom, dof)
plt.figure(figsize=(6,3))
plt.plot(dom, pdf)
plt.show()
```

```
cdf = t.cdf(dom, dof)
plt.plot(dom, cdf)
plt.show()
```


Gamma distribution

https://en.wikipedia.org/wiki/Gamma_distribution

Very general distribution: exponential distribution, Erlang distribution, and chi-square distribution are special cases of the gamma distribution.

Two parameters , both parameters are positive real numbers, can be proposed in two forms:

1. shape k and scale θ.
2. shape α = k and rate β = 1/θ.

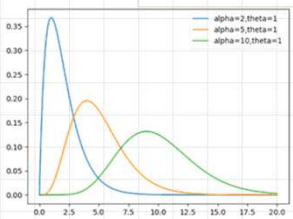
PDF is based on the *gamma function* $\Gamma(x)$, which is an extension of the factorial function to complex numbers (quite complicated: "the gamma function then is defined as the analytic continuation of this integral function to a meromorphic function that is holomorphic in the whole complex plane except zero and the negative integers", *wikipedia*).

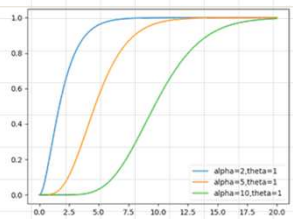
PDF: $f(x) = \frac{\beta^{\alpha} x^{\alpha-1} e^{-\beta x}}{\Gamma(\alpha)}, x, \alpha, \beta > 0$

Gamma distribution

```
from scipy.stats import gamma
dom = np.arange(0, 20, 0.001)
plt.plot(dom, gamma.pdf(dom,2,scale=1), label="alpha=2,theta=1")
plt.plot(dom, gamma.pdf(dom,5,scale=1), label="alpha=5,theta=1")
plt.plot(dom, gamma.pdf(dom,10,scale=1), label="alpha=10,theta=1")
plt.legend()
plt.show()
```

```
plt.plot(dom, gamma.cdf(dom,2,scale=1), label="alpha=2,theta=1")
plt.plot(dom, gamma.cdf(dom,5,scale=1), label="alpha=5,theta=1")
plt.plot(dom, gamma.cdf(dom,10,scale=1), label="alpha=10,theta=1")
plt.legend()
plt.show()
```





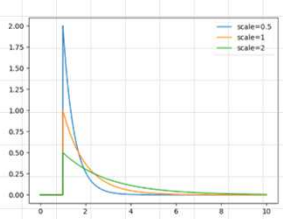
Exponential distribution

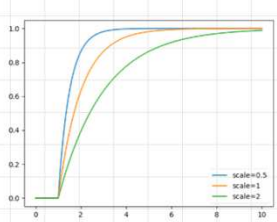
https://en.wikipedia.org/wiki/Exponential_distribution

It has one parameter, λ > 0, called the *rate* parameter.

PDF: $f(x) = \begin{cases} \lambda e^{-\lambda x} & x \geq 0 \\ 0 & x < 0 \end{cases}$

The exponential distribution is primarily used in reliability simulation applications. The exponential distribution in this case is used to model data with a constant failure rate





Exponential distribution

In python, using scipy, parameter $scale = 1/\lambda$

```
from scipy.stats import expon
dom = np.arange(0, 10, 0.001)
plt.plot(dom, expon.pdf(dom,loc=1,scale=0.5), label="scale=0.5")
plt.plot(dom, expon.pdf(dom,loc=1,scale=1), label="scale=1")
plt.plot(dom, expon.pdf(dom,loc=1,scale=2), label="scale=2")
plt.legend()
plt.show()
plt.plot(dom, expon.cdf(dom,loc=1,scale=0.5), label="scale=0.5")
plt.plot(dom, expon.cdf(dom,loc=1,scale=1), label="scale=1")
plt.plot(dom, expon.cdf(dom,loc=1,scale=2), label="scale=2")
plt.legend()
plt.show()
```


Chi squared

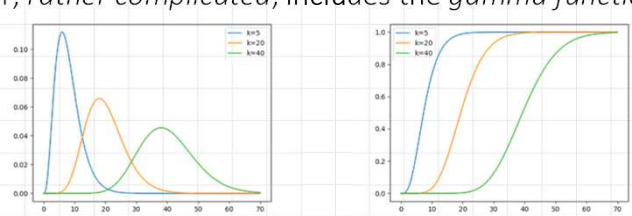
https://en.wikipedia.org/wiki/Chi-squared_distribution

The chi-squared distribution is also used on data drawn from a Gaussian distribution to quantify the uncertainty, for example, for testing the independence of two samples.

The chi-squared distribution has one parameter: degrees of freedom, denoted k.

Usually, as with the Student's t, chi squared is not applied in the direct modeling of natural phenomena; but is used primarily in hypothesis testing.

PDF, *rather complicated*, includes the *gamma function*.



Chi squared

SciPy provides stats.chi2, that can be used to calculate the chi-squared distribution, here for a sample space between 0 and 70 (since the sum squared values must be positive, we need a positive-only sample space).

With 20 degrees of freedom, we can see that the expected value of the distribution is just short of the value 20 on the sample space.

The distribution has a bell-like shape, but it is not symmetric.

```
dom = np.arange(0, 70, 0.01)y
plt.plot(dom, chi2.pdf(dom, 8),label="k=5")
plt.plot(dom, chi2.pdf(dom, 20),label="k=20")
plt.plot(dom, chi2.pdf(dom, 40),label="k=40")
plt.legend()
plt.show()
```

```
plt.plot(dom, chi2.cdf(dom, 8),label="k=5")
plt.plot(dom, chi2.cdf(dom, 20),label="k=20")
plt.plot(dom, chi2.cdf(dom, 40),label="k=40")
plt.legend()
plt.show()
```

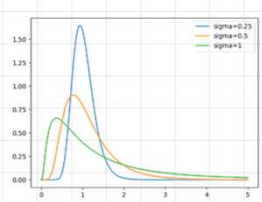

Log-normal distribution

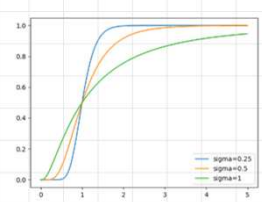
https://en.wikipedia.org/wiki/Log-normal_distribution

The logarithm of the random variable is normally distributed. Much used for models of measurements in exact and engineering sciences, medicine, economics etc. (e.g., energies, concentrations, lengths, financial returns).

PDF: $f(x) = \frac{1}{x\sqrt{2\pi\sigma^2}} e^{-\frac{(\ln(x)-\mu)^2}{2\sigma^2}}$

Parameters are the same as normal.





Log-normal distribution

Using scipy, parameters s (sigma), loc (mu) and scale, where lognorm.pdf(y, s) / scale has y = (x - loc) / scale

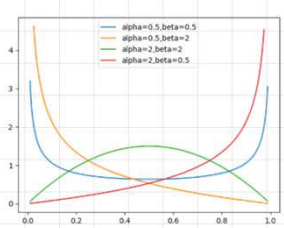
```
from scipy.stats import lognorm
dom = np.arange(0, 5, 0.001)
plt.plot(dom, lognorm.pdf(dom, 0.25), label="sigma=0.25")
plt.plot(dom, lognorm.pdf(dom, 0.5), label="sigma=0.5")
plt.plot(dom, lognorm.pdf(dom, 1), label="sigma=1")
plt.legend()
plt.show()
plt.plot(dom, lognorm.cdf(dom, 0.25), label="sigma=0.25")
plt.plot(dom, lognorm.cdf(dom, 0.5), label="sigma=0.5")
plt.plot(dom, lognorm.cdf(dom, 1), label="sigma=1")
plt.legend()
plt.show()
```

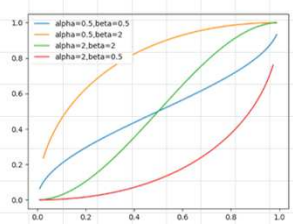

Beta distribution

https://en.wikipedia.org/wiki/Beta_distribution

A continuous probability distributions defined on the interval [0, 1] with two positive shape parameters, α and β. The beta distribution has been applied to model the behavior of random variables limited to intervals of finite length in a wide variety of disciplines.

PDF = $const \cdot x^{\alpha-1}(1-x)^{\beta-1}$, where const comes from the beta function $B(\alpha, \beta)$





Beta distribution

```
from scipy.stats import beta
dom = np.arange(0.01, 0.99, 0.001)
plt.plot(dom, beta.pdf(dom, 0.5,0.5), label="alpha=0.5,beta=0.5")
plt.plot(dom[15:], beta.pdf(dom[15:], 0.5,2), label="alpha=0.5,beta=2")
plt.plot(dom, beta.pdf(dom, 2,2), label="alpha=2,beta=2")
plt.plot(dom[:-15], beta.pdf(dom[:-15], 2,0.5),
label="alpha=2,beta=0.5")
plt.legend()
plt.show()
plt.plot(dom, beta.cdf(dom, 0.5,0.5), label="alpha=0.5,beta=0.5")
plt.plot(dom[15:], beta.cdf(dom[15:], 0.5,2), label="alpha=0.5,beta=2")
plt.plot(dom, beta.cdf(dom, 2,2), label="alpha=2,beta=2")
plt.plot(dom[:-15], beta.cdf(dom[:-15], 2,0.5),
label="alpha=2,beta=0.5")
plt.legend()
plt.show()
```


Discrete distributions

A discrete (univariate) distribution counts occurrences that have countable or finite outcomes: a discrete distribution is constructed from data that can only get a finite or discrete set of values.

This is in contrast to a continuous distribution, where data can get values anywhere on a continuum.

Discrete distributions often involve statistical analyses of "counts" or "how many times" an event occurs.

Uniform distribution

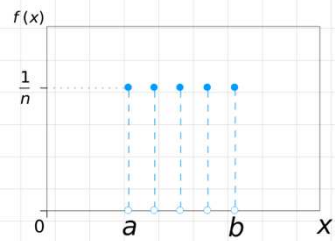
https://en.wikipedia.org/wiki/Discrete_uniform_distribution

(for the continuous counterpart: https://en.wikipedia.org/wiki/Continuous_uniform_distribution)

Simply the distribution where all domain values are taken the same amount of times.

If domain is the interval [a,b]⊂ℑ, where b-a+1=n:

PMF= $f(k) = 1/n$



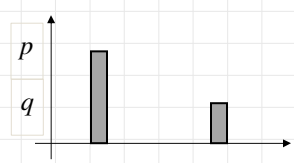
Bernoulli distribution

https://en.wikipedia.org/wiki/Bernoulli_distribution

Only two possible values (say, 0 and 1), one taken with probability p.

PMF: $f(k) = p^k(1-p)^{1-k}$ (equivalently $f(k) = \begin{cases} p & k = 1 \\ q = 1 - p & k = 0 \end{cases}$

Binary classifications fall into this case



Binomial distribution

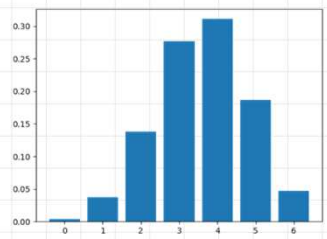
https://en.wikipedia.org/wiki/Binomial_distribution

You repeat n times an experiment that gives a binary result (0/1, yes/no), one taken with probability p (thus, each one is a Bernoulli trial).

The binomial distribution computes the number of successes (say, results 1) over sequences of n trials, thus the probability of getting exactly k successes is given by:

PMF: $f(K, n, p) = \binom{n}{k} p^k (1-p)^{n-k} = \frac{n!}{k!(n-k)!} p^k (1-p)^{n-k}$

```
from scipy.stats import binom
n = 6
p = 0.6
r_values = list(range(n + 1))
dist = [binom.pmf(r, n, p) for r in r_values]
plt.bar(r_values, dist)
plt.show()
```



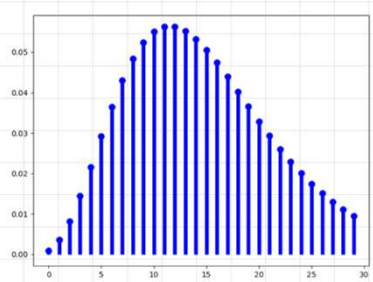
Negative binomial distribution

https://en.wikipedia.org/wiki/Negative_binomial_distribution

Inverse of the binomial: distribution representing the number of Bernoulli trials required to have r number of successes.

Equivalently: number of failures before the binomial experiment results in r successes, or number of successes before the binomial experiment results in r failures.

```
from scipy.stats import nbinom
dom = np.arange(0, 30)
r = 5
P = 0.25
nbinom_pd = nbinom.pmf(dom, r, P)
fig, ax = plt.subplots(1, 1, figsize=(8, 6))
ax.plot(dom, nbinom_pd, 'bo', ms=8)
ax.vlines(dom, 0, nbinom_pd, colors='b', lw=5)
plt.show()
```



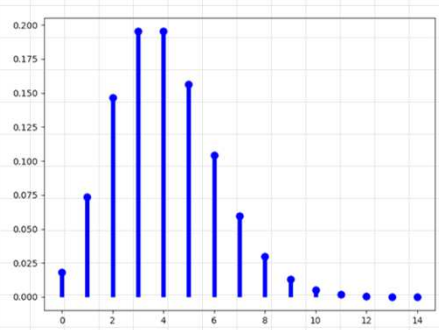
Poisson distribution

https://en.wikipedia.org/wiki/Poisson_distribution

It gives the probability of an event occurring r number of times in a given interval (of time or space).

The events must occur with a known constant mean rate and must be independent of each other. The mean number of occurrences is represented by λ.

Example: assume that the avg number of buses that come on a bus stop in span of 30 minutes is 1. Poisson distribution can model the probability of 0, 1, 2, 3, 4 buses coming to the bus stop within the next 30 minutes.

Poisson distribution

```
from scipy.stats import poisson
dom = np.arange(0, 15)
lmbda = 4
poisson_pd = poisson.pmf(dom, lmbda)
fig, ax = plt.subplots(1, 1, figsize=(8, 6))
ax.plot(dom, poisson_pd, 'bo', ms=8, label='poisson pmf')
ax.vlines(dom, 0, poisson_pd, colors='b', lw=5)
plt.show()
```